

•



- [Core Concepts](#)
- [Workflows](#)
- Time, schedules and timezones

On this page

Time, schedules and timezones

Was this page helpful? 

Timestamps: moments in time

Every event that Pulse receives must carry a timestamp — a specification of *when* the event happened. This is necessary so that Pulse can compute metrics such as — number of events in the last 24 hours — (among other things), since this requires knowing which events happened when.

Timestamps can take several forms. The following are the two most common formats:

- UNIX timestamp, i.e. number of milliseconds (or seconds) since January 1, 1970 00:00 UTC. For example, 1620037816000.

- ISO 8601 timestamp, i.e. a string with date, time and timezone. For example, 2021-05-03T09:49:48+00:00.

Timezones

The same timestamp will correspond to different local time (and sometimes date) depending on the timezone:

Timestamp	Time in UTC (Iceland)	Time in UTC-12 (Baker Island)	Time in UTC+8:45 (Eucla)
1620037816000	Monday, 3 May 2021 10:30:16	Sunday, 2 May 2021 22:30:16	Monday, 3 May 2021 19:15:16
2021-05-03T10:30:16+03:00	Monday, 3 May 2021 07:30:16	Sunday, 2 May 2021 19:30:16	Monday, 3 May 2021 16:15:16

As you will see below, some Pulse operations are timezone-dependent. For example, to “run a job at midnight” Pulse needs to know: which midnight? Midnight in Iceland? Midnight in Eucla? Events that Pulse receives may happen in any timezone anywhere in the world. To schedule jobs and perform other operations consistently, Pulse needs to pick one reference timezone and stick with it. This timezone, which we will call the internal timezone, defaults to the local timezone of the machine where Pulse is running. It can be explicitly configured to a different one via the `pulse.global.timezone` configuration property.

This is similar to dealing with different units of measurement. For example, if you are comparing distances to a threshold of 100 meters, you will convert all measurements to meters – it does not matter if they were originally in miles or in yards. In fact, a timestamp does not necessarily convey *where* the event happened anyway. Just because a timestamp is in UTC does not mean the event happened in UTC – it might have been converted by some intermediate system. Similarly, if a distance is expressed in nautical miles, it does not necessarily mean it was measured at sea.

Changing the default value of the internal timezone can be advantageous. For example, consider a deployment for an Australian client where the system architect wants the jobs to run at midnight local time, when the system is under the lowest load. If the machines are not in Australia, the default timezone might be UTC – in this case the job execution time needs to be set to 10 PM. It is much more intuitive though to configure the internal timezone to UTC+10 and set the job execution time to 0 AM.

A word of caution: some timezones, such as “Europe/Lisbon” (as opposed to “WEST” or “UTC+1”), automatically incorporate daylight saving time. In these timezones, the time of 1:30 AM, for example, might be observed twice on the same day (in October) or not at all (in March). For this reason it is not recommended to use such timezones as the internal timezone. This is another reason to explicitly set the internal timezone instead of relying on the default.

Workflow clock

Pulse performs many time-based operations, for example:

- Compute a real-time metric (see [Plans in workflows - Metrics](#)) such as “number of events since the start of the day”
- Run a scheduled metrics job (see [Scheduled Metrics Jobs](#)) at midnight.
- Enable a list item (see [Lists](#)) on 2021-05-03 at 15:00.

From the section above, you already know that this time is relative to the reference timezone. But is it *the time* as we know it? Not quite.

Unlike in the outside world, **time in Pulse only moves when Pulse receives events**. For example, if Pulse needs to enable a list item at 15:00, it will not do so until it receives an event that happened after 15:00 (even if it has to wait for days). This allows Pulse to correctly process events if they are consistently late. For example, a bank might only send information about transfers made during the weekend on Monday morning. In this case, when processing each transfer, Pulse “pretends” that it is still the weekend.

It’s also important that (under normal circumstances) **time in Pulse only moves forward**. While Pulse is designed to handle consistently delayed events correctly (per above), it does not retroactively process late events if more recent events have arrived in the meantime.

Instead, Pulse processes these “out of order” events as if they were recent. This is a limitation of how Pulse computes real-time metrics, but it affects the logic for the entire workflow.

Here is how this works in practice. The current time in Pulse is stored in the **workflow clock**.

- When there are no events, the clock does not change.
- Each new event, if it's more recent than the workflow clock, will advance the clock. For example, if the clock is at 1620037816000 (10:30 UTC) and there is a new event with a timestamp of 09:40 UTC-1 (i.e. 10:40 UTC), the clock will advance to 1620038416000 (10:40 UTC).
- If a new event is older than the clock, Pulse will override its timestamp to equal the clock. The clock itself remains the same.

How workflow clock and internal timezone affect Pulse functionality

In this section you will see how Pulse clock and the internal timezone affect different functionality of Pulse.

Lists

Pulse [Lists](#) use the workflow clock to enable and disable items. For example, if an item needs to be enabled on 2021-05-03 at 10:30 UTC, it will happen when the workflow clock reaches that moment in time (i.e. when Pulse receives an event with a timestamp higher or equal to 2021-05-03 10:30 UTC).

Real-time metrics

Real-time metrics (see [Plans in workflows - Metrics](#)) use the workflow clock to compute aggregations over time periods. For example, in a metric like "total amount transacted by customer in the last 24 hours", 24 hours are counted backwards from the workflow clock, i.e. the latest event received by Pulse.

Real-time metrics may also rely on the internal timezone. For example, a metric with a window expression *reset (_: day)*, which means the metric is reset every day at midnight, will depend on Pulse's notion of midnight. If Pulse timezone is set to UTC, then the metric will reset at midnight UTC. (Remember, the timezone of where the events actually happened is irrelevant.)

Scheduled metrics jobs

[Scheduled Metrics Jobs](#) can be triggered in two ways:

- Automatically, based on their schedule (e.g. "run every midnight").
- Manually, using a button in the Pulse UI.

In both cases, Pulse uses the workflow clock, but the behavior is different. There are two main questions:

- When does the job start?
- What time period does the job consider?

Here is a quick summary, assuming the job computes metrics with windows of 2 days.

Job triggered...	Job starts...	Time period
Automatically on schedule (<i>"every midnight"</i>)	When workflow clock reaches "midnight" in internal timezone .	Events with timestamps within 2 days of "midnight" in internal timezone .
Manually (through the UI)	Immediately	Events with timestamps within 2 days of the current workflow clock .

Below are a few important considerations based on this behavior.

Automatically triggered jobs

Keep in mind when thinking about when the job starts:

- The job is triggered based on the workflow clock, not real-world clock. If it's 1 AM in the real world, but the last event received has a timestamp of 23:59, **the job will not trigger until a more recent event is received**.
- **If Pulse does not receive events on a given day, the job for that day will never trigger.** If Pulse only receives events on April 30 and then only on May 3, the first event of April 30 will trigger the job for that day, and the first event of May 3 will trigger the job for that day. No jobs will run for May 1 or May 2.

- In case of Omnichannel scheduled metrics (see [Plans in workflows - Omnichannel metrics](#)), there are multiple workflows involved: the main workflow and one or more secondary workflows. In this case Pulse uses the **clock of the main workflow**. The same considerations as above apply. For example, **if there are no events on the main channel, the job will not trigger**, no matter how many events are received on the secondary channels.

Keep in mind when thinking about the time period considered by job:

- The time period for the job is calculated as follows: time when the job is supposed to trigger + biggest metric window size. Note again that the time period is not counted from current value of the workflow clock, but from the time the job is scheduled to run (e.g. midnight).
- The time the job is scheduled to run (e.g. midnight) is interpreted according to the internal timezone.
- In case of Omnichannel scheduled metrics (see [Plans in workflows - Omnichannel metrics](#)), **the same time period is used consistently across all channels**. If the main workflow clock is on May 3, the job will correspond to midnight of May 3 and the time period will be counted from that moment in all channels. It does not matter if some of the secondary workflows, for example, have not received events since May 2 or, to the contrary, have already received events from May 4.

Example:

Workflow clock	Job schedule	Window size	Time period
May 3, 01:30 AM	midnight (0 AM)	2 days	May 1, 0 AM – May 3, 0 AM

Manually triggered jobs

- The time period is until the timestamp of last event received. If you trigger a job on Monday but the last received event was on Friday, the time period will be until Friday, not Monday.
- In Omnichannel scheduled metrics, the time period is until the timestamp of the last event received on the main channel. Like with automatic jobs, the same time period is applied consistently across all channels.

Example:

Workflow clock	Job schedule	Window size	Time period
May 3, 10:30 AM	midnight (0 AM)	2 days	May 1, 10:30 AM – May 3, 10:30 AM